3/10 Testing Ágil (debrief)

Resumen Ejecutivo

TÍTULO: El Testing Proactivo en Ambientes Ágiles: Calidad Integrada y Retroalimentación Continua.

El *Testing Ágil* representa un cambio fundamental del enfoque tradicional (cascada), pasando de ser una fase tardía y secuencial a una **actividad continua, paralela y temprana** al desarrollo. Su objetivo principal es proveer **retroalimentación (*feedback*) rápida** al equipo para guiar las decisiones y minimizar el costo de los defectos, ya que un error detectado tempranamente es significativamente más económico de corregir.

El Pilar del Desarrollo Conducido por Pruebas (TDD)

El principio de **Desarrollo Conducido por Pruebas (TDD)** es la práctica ágil más influyente en la calidad del código (Q1). TDD requiere escribir primero una **prueba unitaria** que falle, luego el código para que esta pase, y finalmente **refactorizar** el código para mantener su limpieza y mantenibilidad. Esta práctica, junto con las **Pruebas de Integración y Regresión Automatizadas**, forma la base robusta de la **Pirámide de Testing**, asegurando que la mayor parte del esfuerzo se concentre en pruebas rápidas y económicas.

La Responsabilidad Compartida y los Cuadrantes

En ambientes ágiles, la calidad es una **responsabilidad compartida** por todo el equipo (*Whole-Team Approach*). Los *testers* dejan de ser "policías de calidad" para convertirse en **colaboradores expertos**. El *framework* de los **Cuadrantes de Testing Ágil** ayuda al equipo a asegurar una cobertura completa, dividiendo el *testing* en:

- **Q1 (Tecnología/Soporte):** Pruebas Unitarias y de Componente (Calidad Interna).
- **Q2 (Negocio/Soporte):** Pruebas de Historia y Aceptación (*ATDD*) (Calidad Externa y Requisitos).
- **Q3 (Negocio/Crítica):** Pruebas Exploratorias y Usabilidad (Descubrimiento de lagunas).
- **Q4 (Tecnología/Crítica):** Pruebas de Carga/Rendimiento y Seguridad (Riesgos No Funcionales).

Automatización, Desafíos y Vínculos

La **automatización** es esencial, ya que reduce el costo de la regresión, incrementa la cobertura y proporciona la **red de seguridad** necesaria para la **Integración Continua (CI)** y la **Refactorización** constante.

La transición al *testing* ágil es un **desafío cultural** que requiere que los *testers* adquieran **habilidades técnicas** (automatización) y una mentalidad proactiva, participando activamente desde la definición de los requerimientos junto con el cliente. La adhesión a principios ágiles y el uso de métricas de *testing* se consideran la mejor medida de progreso para la entrega continua de *software* de valor.

Contexto

Síntesis: El testing en el desarrollo de software es una actividad transversal que busca la verificación y validación del producto. En el contexto moderno, impulsado por metodologías ágiles, el testing se transforma de una fase secuencial y tardía a una actividad continua y colaborativa que impulsa el desarrollo.

Explicación Clara y Definiciones:

El testing es una investigación técnica para proveer información a los *stakeholders* sobre la calidad del producto o servicio bajo pruebas. Su objetivo principal es encontrar defectos y aportar calidad.

- **Verificación:** Asegura que el *software* está siendo construido de acuerdo con los estándares y las especificaciones.
- **Validación:** Asegura que el *software* satisface los requerimientos del cliente y es apto para el propósito (cumple con las necesidades del cliente).

El testing es una actividad **destructiva** (busca fallas), pero su aporte final es **completamente constructivo**.

Testing en Enfoque Tradicional vs. Testing en Ambientes Ágiles

Concepto	Enfoque Tradicional (Casada / Definido)	Enfoque Ágil (Empírico)
Visión del Proceso	Proceso definido; busca la predictibilidad.	Proceso empírico; basado en ciclos cortos de inspección y adaptación.
Rol del Testing	Una **fase** separada, generalmente al final (lineal al desarrollo).	Una **actividad continua y paralela** al desarrollo. No es una fase.
Énfasis de Calidad	Conformidad estricta con los requerimientos especificados *up front*.	**Entrega de valor** y *software* funcionando.
Gestión de Riesgo	El testing es una "red de seguridad" que intenta atrapar errores tardíamente.	El testing temprano (testing *first*) está enfocado a **reducir riesgos** desde el inicio.
Documentación	Alta dependencia de documentos detallados (*Especificación de Requerimientos*) para planificar y escribir casos de prueba.	Los casos de prueba y los ejemplos de aceptación a menudo **reemplazan los documentos de requisitos detallados**.
Objetivo Primario	Asegurarse de que *todos* los requisitos especificados se entreguen. Un retraso es común si no se cumplen hitos.	**Retroalimentación rápida** (*quick feedback*) que guía el desarrollo.
Triple Restricción	Alcance fijo; Tiempo y Costo se ajustan.	Tiempo y Recursos fijos (iteración); **Alcance variable**.

En el ambiente tradicional, si el equipo de pruebas no recibe el *software* hasta el final del proyecto, es como "pedir un examen de sangre cuando el paciente ya está muerto". En ágil, las actividades ocurren **concurrentemente** y los plazos se acortan significativamente.

Testing Manifesto / Valores Testing Manifiesto

El *Testing Manifesto* establece los valores y principios que guían la mentalidad de los *testers* en ambientes ágiles.

Valores y Mentalidad del Tester Ágil (Agile Testing Mind-Set):

El *tester* ágil es un profesional que adopta el cambio, colabora con personas técnicas y de negocio, y entiende que las pruebas documentan requisitos y dirigen el desarrollo. Esta mentalidad se basa en los siguientes principios:

- **Proveer Retroalimentación Continua:** La retroalimentación (*feedback*) impulsa los proyectos ágiles. El *tester* es un proveedor inherente de información y debe ayudar a la dueña del producto (*Product Owner*) a articular los requisitos como ejemplos y pruebas ejecutables.
- **Entregar Valor al Cliente:** La prioridad es la satisfacción del cliente mediante la entrega temprana y continua de *software* valioso.
- **Colaboración y Comunicación Cara a Cara:** La comunicación entre desarrolladores, *testers* y clientes es el medio más eficaz. Esto ayuda a reducir la latencia del *feedback*.
- **Tener Coraje:** Coraje para refactorizar y hacer cambios, apoyados por una red de seguridad de pruebas de regresión automatizadas. También, coraje emocional para pedir ayuda o señalar obstáculos.
- **Mantenerlo Simple:** Buscar el enfoque más simple que funcione. Esto se relaciona con el principio de simplicidad ágil: maximizar la cantidad de trabajo no realizado.
- **Práctica de Mejora Continua (*Continuous Improvement*):** A través de retrospectivas, el equipo evalúa su desempeño y ajusta su forma de trabajar.
- **Responder al Cambio:** Los *testers* deben acoger el cambio, aunque la estabilidad sea tradicionalmente buscada. La respuesta al cambio es un valor clave.
- **Auto-Organizarse (*Self-Organize*):** El equipo es responsable de auto-organizar y gestionar sus propias cargas de tareas.

9 Principios para Testing en Proyectos Ágiles

1. **Testing se mueve hacia adelante en el proyecto:** Las pruebas deben comenzar lo antes posible, incluso desde que se definen los requerimientos. El objetivo es verificar que el requerimiento es testeable.
2. **Testing no es una fase:** El testing es una actividad que acompaña y retroalimenta al desarrollo desde el principio.
3. **Todos hacen testing:** La calidad es responsabilidad de todo el equipo. Los roles se difuminan (*blur the lines*) y cualquiera puede asumir tareas de *testing*.
4. **Reducir la latencia del feedback:** El ciclo de *feedback* debe ser lo más corto posible. Si el *feedback* es lento, el proceso de desarrollo y la localización de fallos se vuelven costosos.
5. **Las pruebas representan expectativas:** Los casos de prueba (especialmente los de aceptación) definen las expectativas de negocio del cliente.
6. **Mantener el código limpio, corregir los defectos rápido:** Al usar prácticas como Integración Continua (CI), si una prueba falla, el error debe corregirse de inmediato para evitar la acumulación de deuda técnica.
7. **Reducir la sobrecarga de documentación de las pruebas:** Se prioriza la comunicación y las interacciones sobre la documentación extensa. El *software* funcionando es la medida de progreso.
8. **Las pruebas son parte del "done" (Definición de Hecho):** Para que una historia de usuario se considere "hecha" (DoD), debe haber pasado por las pruebas requeridas, a menudo incluyendo pruebas unitarias automatizadas.
9. **De probar al final a Conducido por Pruebas (Test-Driven):** El enfoque cambia a escribir pruebas antes de escribir el código (TDD/ATDD), lo cual previene errores y guía el diseño.

6 Prácticas Concretas de Testing Ágil

Las prácticas concretas se centran en la automatización y la colaboración para garantizar la calidad del *software*.

1. **Pruebas de unidad e integración automatizadas:**
   - **Pruebas de Unidad (*Unit Tests*):** Prueban componentes individuales (métodos o clases de objetos). Son realizadas por el desarrollador.
   - **Pruebas de Componente:** Verifican el comportamiento de una parte más grande del sistema, como un grupo de clases que proveen un servicio.
   - La automatización de estas pruebas es esencial para la agilidad, permitiendo la ejecución rápida y frecuente, y formando la base de la pirámide de *testing*.
2. **TDD (Test Driven Development) - Desarrollo Conducido por Pruebas:**
   - **Definición:** Técnica avanzada que guía el desarrollo al escribir primero la prueba (*Test First Development*) y luego refactorizar.
   - **Proceso (Red-Green-Refactor):** 1. Escribir una prueba que falle (Red). 2. Escribir el código mínimo para que pase (Green). 3. Reestructurar y limpiar el código (*Refactoring*) sin cambiar el comportamiento externo.
   - **Beneficios:** TDD obliga a modularizar el código, garantiza que el código sea testeable y mantenible, y previene errores al forzar la comprensión de la especificación antes de codificar.
3. **Pruebas de Regresión a nivel de sistema automatizadas:**
   - **Regresión:** Consiste en volver a ejecutar pruebas que se completaron con éxito después de realizar cambios, para garantizar que el nuevo código no introduzca errores o *bugs*.
   - La automatización **reduce drásticamente los costos** y el tiempo de la regresión, lo que es fundamental para la integración continua y la refactorización frecuente.
4. **ATDD (Desarrollo conducido por Pruebas de aceptación):**
   - **Definición:** El cliente (o *Product Owner*) especifica las pruebas de aceptación, a menudo en colaboración con el equipo, para definir si una historia de usuario cumple con sus expectativas.

- Estas pruebas de aceptación pueden reemplazar la documentación detallada de requisitos. Son pruebas de validación que se centran en el valor de negocio.

5. **Pruebas exploratorias (*Exploratory Testing*):**
   - **Definición:** Es una estrategia en la que el **diseño y la ejecución de las pruebas ocurren simultáneamente**. Se basa en la intuición, conocimiento, creatividad y experiencia del *tester*.
   - Se utiliza para obtener información rápidamente, para revelar nuevos tipos de defectos y es muy efectiva, especialmente en código legado, o cuando el *tester* no escribió el código.
6. **Control de versión de las pruebas con el código:**
   - Las pruebas automatizadas son artefactos clave que deben ser gestionados bajo control de configuración (*SCM*) y versionados junto con el código de producción que verifican. Esto asegura la trazabilidad y la integridad de la configuración.

El rol del Tester en los equipos ágiles

El rol del *tester* en un equipo ágil es intrínsecamente diferente al modelo tradicional, debido al **enfoque de equipo completo (*whole-team approach*)** donde la calidad es responsabilidad compartida.

- **Colaborador Integral:** El *tester* trabaja estrechamente con el cliente/PO para definir los requisitos (historias) en forma de pruebas ejecutables (Q2).
- **Defensor de la Calidad (*Advocate for Quality*):** El *tester* aboga por la calidad en nombre del cliente y ayuda al equipo a entregar el máximo valor.
- **Habilidades Técnicas:** Se espera que el *tester* tenga buenas habilidades técnicas y ayude a automatizar pruebas o a colaborar con los desarrolladores en ello. Los *testers* que solo hacen pruebas manuales de interfaz de usuario podrían tener problemas para adaptarse a este ambiente automatizado.
- **Mentalidad de Mejora Continua:** El *tester* busca herramientas y prácticas que ayuden a añadir más valor y a obtener un mejor retorno de la inversión para el cliente.

Desafíos culturales y Cambios culturales

La transición al *testing* ágil es, ante todo, un proceso de **cambio cultural**.

- **Barreras de Adopción:**
  - **Miedo y Pérdida de Identidad:** Los *testers* pueden temer perder su identidad o que su rol se devalúe si reportan a un gerente de desarrollo.
  - **Mentalidad "Quality Police":** Los *testers* acostumbrados a ser "policías de calidad" que protegen el código se enfrentarán a la resistencia, ya que deben pasar a ser colaboradores.
  - **Falta de Habilidades y Formación:** Muchos *testers* carecen de las habilidades de programación y automatización necesarias para el enfoque ágil.
  - **El "Mini-Waterfall":** Los equipos nuevos a menudo caen en el patrón de "mini-cascada", donde el desarrollo se realiza primero y el *testing* se realiza al final de la iteración, repitiendo el problema de la lentitud del *feedback*.
- **Estrategias para el Cambio:**
  - **Transparencia y Apoyo:** Los gerentes deben proporcionar recursos, tiempo y entrenamiento (a menudo con un *coach* experimentado).
  - **Empoderar al Equipo (*Empower Your Team*):** El equipo debe sentirse libre para tomar decisiones y experimentar creativamente.
  - **Inclusión Temprana:** Los *testers* deben participar en discusiones de diseño y planificación (*stand-ups*, reuniones de planificación), y no esperar la entrega de código.
  - **Enfoque de "Ser Ágil" (*Being Agile*):** Esto requiere un cambio de mentalidad a nivel organizacional, no solo practicar rituales ágiles (*Doing Agile*).

Cuadrantes de Testing Ágil

Los **Cuadrantes de Testing Ágil de Brian Marick** son una herramienta para categorizar los diferentes tipos de pruebas necesarias para garantizar la calidad en un proyecto ágil. Utilizan dos ejes: **Orientado al Negocio (*Business-Facing*) vs. Orientado a la Tecnología (*Technology-Facing*)** y **Soporte al Equipo (*Support the Team*) vs. Crítica del Producto (*Critique the Product*)**.

Eje	Soporte al Equipo (Q1, Q2)	Crítica del Producto (Q3, Q4)
Orientado a la Tecnología (Q1, Q4)	**Q1: Pruebas para guiar el diseño.** Pruebas Unitarias, Pruebas de Componente. **Foco:** Calidad interna (mantener el código limpio). Automatizadas.	**Q4: Pruebas para mitigar el riesgo técnico.** Pruebas de Performance/Carga, Pruebas de Seguridad, Pruebas de "ilidades" (*-ilities* como fiabilidad, escalabilidad, etc.). **Foco:** Requisitos No Funcionales (RNF). Utilizan herramientas automatizadas.
Orientado al Negocio (Q2, Q3)	**Q2: Pruebas para guiar la aceptación.** Pruebas Funcionales, Pruebas de Historia (*Story Tests*), Ejemplos, Prototipos, Simulaciones. **Foco:** Calidad externa, definir funcionalidad deseada. Automatizadas y manuales.	**Q3: Pruebas para descubrir lagunas y problemas.** Pruebas Exploratorias, Pruebas de Usabilidad, UAT (*User Acceptance Testing*), Pruebas Alfa/Beta. **Foco:** Usabilidad, valor de negocio, experiencia del usuario. Principalmente manuales.

Uso: Los cuadrantes actúan como una lista de verificación (*checklist*) para asegurar que se cubren todos los aspectos de la calidad del *software*.

Pirámide de Testing Agile

Síntesis: La Pirámide de Testing representa la estrategia ágil ideal para la automatización de pruebas, donde la mayor parte del esfuerzo se concentra en las pruebas rápidas, baratas y de bajo nivel (pruebas unitarias), minimizando las pruebas lentas y costosas de alto nivel.

Comparación con el Enfoque Tradicional:

Nivel de Prueba	Pirámide Ágil	Pirámide Tradicional (Implícita/Invertida)
Base (Ancha)	**Pruebas Unitarias** y de Componente. Gran cantidad, muy rápidas y automatizadas (Q1).	**Pruebas de Interfaz de Usuario/Sistema** (lentas y costosas). Concentración de esfuerzo manual en la parte superior.
Medio	Pruebas de Servicio/Integración (automatizadas Q2).	Pruebas de Integración (menor esfuerzo que la cima).
Cima (Estrecha)	Pruebas de Sistema/Interfaz de Usuario (UAT, Exploratorias Q3). Pequeña cantidad, enfocada en la experiencia y el riesgo.	Pruebas Unitarias/Componente (menor esfuerzo o nulo).

Implicación: Concentrar el esfuerzo en la base (Pruebas Unitarias) permite un **ciclo de *feedback* más rápido**, **menor costo** de localización de defectos y una **mayor confianza** para refactorizar. Si la pirámide se invierte (más pruebas en la cima), el equipo se ralentiza y el costo de los defectos aumenta.

Niveles de Prueba

Los niveles de prueba se definen por la granularidad y el objetivo:

- **Pruebas de Unidad (*Unit Tests*):** Nivel más bajo. Prueban el correcto funcionamiento de módulos, métodos o clases individuales. Se realizan teniendo acceso al código fuente y se espera que se reparen los errores tan pronto como se encuentran.
- **Pruebas de Integración (*Integration Tests*):** Verifican que los componentes del sistema, que funcionan bien aisladamente, también lo hacen en conjunto (ej. interacciones de clases a través de sus interfaces). Se busca verificar la funcionalidad de dos o más componentes juntos.
- **Pruebas de Sistema (*System Tests*):** Prueban el sistema parcial o completo para verificar que cumple con sus requerimientos. Las Pruebas de Carga/Rendimiento y las Pruebas de Seguridad (Q4) son ejemplos típicos de pruebas de sistema.
- **Pruebas de Aceptación (*Acceptance Tests* / UAT):** Pruebas que realiza el usuario (o el PO en Scrum) para verificar que el producto cumple con lo solicitado y se ajusta a sus necesidades. Su meta principal es

establecer confianza en el sistema, no encontrar defectos. Incluyen:

- **Pruebas Alfa:** Pruebas realizadas por el usuario en ambiente de laboratorio.
- **Pruebas Beta:** Pruebas en ambientes de trabajo reales (clientes potenciales usan el sistema y reportan problemas).

Beneficios de la automatización del testing

La automatización es un componente esencial de la agilidad que soporta la Integración Continua (CI) y la capacidad de responder al cambio.

1. **Rápida ejecución y Reducción de costos:** Permite la ejecución rápida de suites de pruebas, lo que reduce el tiempo y el esfuerzo de la regresión manual.
2. **Reusabilidad y Mayor cobertura/alcance:** Los casos de prueba automatizados pueden correrse en múltiples configuraciones (OS, navegadores, DB) sin duplicar el costo, expandiendo dramáticamente el alcance de las pruebas con el mismo costo.
3. **Mayor precisión:** Reduce los errores humanos que pueden ocurrir durante las tareas repetitivas de prueba manual.
4. **Red de Seguridad (*Safety Net*):** Proporciona la confianza necesaria para que los desarrolladores puedan **refactorizar constantemente** y hacer cambios sin introducir nuevos *bugs* (regresión).
5. **Documentación Viva:** Las pruebas automatizadas que ilustran el comportamiento deseado se convierten en documentación que refleja cómo funciona el sistema.
6. **Moral del Tester:** Disminuye las tareas rutinarias, permitiendo al *tester* concentrarse en tareas intelectualmente desafiantes, como el *testing* exploratorio.

TDD (Test Driven Development)

TDD

El **Desarrollo Guiado por Pruebas (TDD)** es el acto de diseñar *tests* que previene errores y guía el diseño.

Definición: TDD es una técnica avanzada que fusiona dos prácticas fundamentales: **Escribir las pruebas primero (*Test First Development*)** y **Refactorización (*Refactoring*)**.

- **Test First Development:** Se escriben las pruebas unitarias antes de cualquier código funcional. Dado que estas pruebas prueban unidades de código concretas, obliga a los desarrolladores a **modularizar** el código.
- **Refactorización:** Consiste en reestructurar el código existente para mejorar su claridad y reducir su complejidad (mejora atributos no funcionales) **sin cambiar su comportamiento externo**.

El Corazón de TDD

El proceso central de TDD (Red-Green-Refactor) es un ciclo incremental donde se:

1. Identifica la nueva funcionalidad.
2. Se escribe una prueba automatizada para esa funcionalidad que **falla deliberadamente** (Red).
3. Se implementa la funcionalidad con el código mínimo (Green).
4. Se corren todas las pruebas existentes, incluyendo la nueva, hasta que todas pasan.
5. Se realiza la **refactorización** del código.
6. Se avanza a la siguiente funcionalidad.

Las 3 leyes de TDD

1. **No escribir una línea de código hasta que no hayas escrito antes un *test case* que falla**.
2. No encontrado en fuentes. (Nota: La fuente proporciona un título para "Las 3 leyes de TDD" pero solo desarrolla la primera ley explícitamente. La práctica implica la segunda: escribir código mínimo; y la tercera:

refactorizar.)

3. No encontrado en fuentes.

Secciones Transversales

Glosario Crítico

Concepto	Definición y Fuente	Vínculo
Testing Ágil	Estilo de trabajo que se centra en la retroalimentación continua y la colaboración. No es una fase, sino una actividad paralela y continua al desarrollo.	Opuesto al enfoque de cascada; basado en el empirismo.
TDD (*Test Driven Development*)	Técnica avanzada donde se escribe primero una prueba unitaria que falla, luego el código para hacerla pasar, y finalmente se refactoriza.	Práctica central de Q1 (Tecnología/Soporte al Equipo). Mejora la calidad y el diseño del código.
ATDD (*Acceptance Test Driven Development*)	Desarrollo guiado por pruebas de aceptación definidas por el cliente para asegurar que el *software* satisfaga las necesidades del negocio.	Base de Q2 (Negocio/Soporte al Equipo).
Pirámide de Testing	Representación jerárquica que aconseja concentrar la mayoría del esfuerzo de prueba en la base (Pruebas Unitarias automatizadas) para obtener *feedback* rápido y reducir costos.	Estrategia de automatización clave en ágil.
Pruebas Exploratorias	Estilo de *testing* en el que el aprendizaje, el diseño y la ejecución de pruebas se realizan simultáneamente, dependiendo de la habilidad y creatividad del *tester*.	Típico de Q3 (Negocio/Crítica del Producto).
WIP Limit (Trabajo en Progreso)	No encontrado en fuentes como concepto desarrollado.	*Nota:* Aunque se mencionó en el contexto de Lean/Kanban en la conversación previa, no está desarrollado en las fuentes actuales sobre *Agile Testing*.

Cuadros Comparativos: Roles y Enfoques

Aspecto	Rol del Tester en Tradicional	Rol del Tester en Ágil
Responsabilidad de Calidad	Asume la propiedad de la calidad (el "policía de calidad").	Propiedad compartida por todo el equipo (*Whole-Team Approach*).
Interacción con Devs	Adversarial o de silo; espera la entrega del código.	Colaborador activo en el diseño y co-responsable de la automatización.
Timing	Tareas secuenciales, espera la documentación y el producto.	Tareas concurrentes; participa en la definición de requisitos con el cliente.
Habilidades Foco	Documentación de casos de prueba, ejecución manual (típico).	Automatización, Exploratorio, *Coaching* a clientes sobre requisitos (ATDD).

Checklist de Buenas Prácticas en Testing Ágil

Las siguientes prácticas son fundamentales para el éxito del *testing* en un ambiente ágil:

- **Integración Continua (CI):** Usar un servidor de CI que ejecute automáticamente todas las pruebas unitarias y de integración después de cada *check-in* (o commit) para garantizar *feedback* rápido sobre la salud del código.
- **Test-First:** Adoptar TDD y/o ATDD para guiar el desarrollo con pruebas desde el inicio.
- **Inversión en la Base:** Seguir la Pirámide de Testing, maximizando la automatización de pruebas unitarias y de componentes (Q1).

- **Definición de Hecho (DoD):** Incluir las pruebas unitarias y de aceptación automatizadas como requisito obligatorio para que una historia se considere completa.
- **Experimentación:** Utilizar las iteraciones cortas para probar nuevas herramientas o procesos de *testing* y medir su efectividad (*fail fast*).
- **Priorización Basada en Riesgos:** Utilizar la evaluación de riesgos para priorizar qué probar primero y dónde enfocar el mayor esfuerzo.
- **No ser el "Gatekeeper":** Evitar ser el cuello de botella que bloquea el progreso; en su lugar, facilitar la información para que el equipo tome decisiones y se adapte.

Mapa de Relaciones Conceptuales (Vínculos con Otras Unidades)

Unidad Relacionada	Concepto de Testing Ágil	Vínculo Detallado
Gestión de Proyectos (Scrum/XP)	**Iteraciones cortas y DoD.**	El *testing* debe realizarse en paralelo para asegurar que el código desarrollado esté "testeado, y potencialmente desplegable" al final de cada iteración.
Calidad de Software (SQA/QC)	**TDD y Refactorización.**	TDD mejora la calidad intrínseca del código. La refactorización mejora atributos no funcionales como la mantenibilidad. La calidad se **embebe** desde el principio.
Despliegue (DevOps/CI/CD)	**Automatización y Quick Feedback.**	La integración continua (*Continuous Integration*) utiliza la automatización (especialmente Q1) para verificar la compilación y el código, lo que es un requisito para la **Entrega Continua** (*Continuous Delivery*).
Gestión de Configuración (SCM)	**Control de Versión de Pruebas.**	Las pruebas automatizadas (junto con el código fuente y *scripts*) son ítems de configuración que deben ser versionados en el repositorio para asegurar la trazabilidad y la regresión.
Ingeniería de Requerimientos	**ATDD / Pruebas de Aceptación.**	Los requisitos del cliente se articulan como pruebas ejecutables (Q2), reemplazando la documentación formal extensa y asegurando que se entregue el valor deseado.